OAuth2 et OpenID Connect (OIDC)

Spring Security supporte **OAuth2 / OIDC** pour permettre à ton application d'utiliser un **serveur d'authentification externe** comme **Keycloak**, **Google**, **GitHub**, etc.

🔑 A. Les Rôles en OAuth2 (4 acteurs principaux)

Rôle	Description
Resource Owner	L'utilisateur final (celui qui possède les données)
Client	L'application qui veut accéder aux données (ex: ton Spring Boot app)
Authorization Server	Serveur qui gère l'authentification (ex: **Keycloak**, Google, etc.)
Resource Server	L'API sécurisée (ex: ton backend Spring Boot) qui valide le token

🧾 Tokens : Access Token vs ID Token

Type de Token	But	Contenu
Access Token	Permet d'accéder aux APIs	scopes, roles, expiration
ID Token	Donne des infos sur l'utilisateur (OpenID Connect)	nom, email, sub (user ID), etc.

🧭 D. Scopes, Roles et Authorities

1. ✅ Scope (portée d'accès)

Un **scope** est une **permission technique** qu'un client demande lorsqu'il veut accéder à certaines ressources.

🔸 Exemple de scopes :

- `openid` → indique qu'on veut une **authentification via OpenID Connect**
- `email` → on veut l'adresse email de l'utilisateur
- `profile` → on veut son nom, prénom, etc.

➡️ Les scopes sont inclus dans l'Access Token ou ID Token.

```
"scope": "openid email profile"
```

---

2. ✅ Roles (dans Keycloak)

Les **rôles** sont définis dans Keycloak et **assignés à des utilisateurs**, soit :

- au niveau **realm** → `realm roles`
- au niveau **client** → `client roles`

🔸 **Exemple dans Keycloak :**

- Realm Role: `ADMIN`
- Client Role: `user-service:MANAGER`

> Keycloak ajoute ces rôles dans le token JWT, souvent dans le champ :

```
"realm_access": {
  "roles": ["ADMIN", "USER"]
}
```

ou bien :

```
"resource_access": {
  "my-client": {
    "roles": ["MANAGER", "SUPPORT"]
  }
}
```

---

3. ✅ Authorities (Spring Security)

En Spring Security, **les authorities sont les rôles et permissions** que Spring attribue à l'utilisateur après authentification.

👉 Pour que Spring comprenne les rôles envoyés par Keycloak, il faut **mapper** les rôles Keycloak en `GrantedAuthority`.

Par convention Spring :

- `"ADMIN"` devient `"ROLE_ADMIN"`
- `"MANAGER"` devient `"ROLE_MANAGER"`

🔄 Exemple Complet d'un JWT de Keycloak

```
{
  "sub": "user-id",
  "preferred_username": "john.doe",
  "scope": "openid email profile",
  "realm_access": {
    "roles": ["ADMIN", "USER"]
  },
  "resource_access": {
    "my-client": {
      "roles": ["MANAGER"]
    }
```

```
  }
}
```

🛠️ **Mapper les rôles Keycloak → Authorities Spring**

Tu dois créer un `JwtAuthenticationConverter` personnalisé pour lire `realm_access.roles` ou `resource_access.my-client.roles` et les transformer en `ROLE_XXX`.

🔧 **Exemple :**

```
@Bean
public JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtAuthenticationConverter converter = new
JwtAuthenticationConverter();
    converter.setJwtGrantedAuthoritiesConverter(jwt -> {
        List<String> roles = jwt.getClaim("realm_access") != null
            ? ((Map<String, List<String>>)
jwt.getClaim("realm_access")).get("roles")
            : Collections.emptyList();

        return roles.stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" + role))
            .collect(Collectors.toList());
    });
    return converter;
}
```

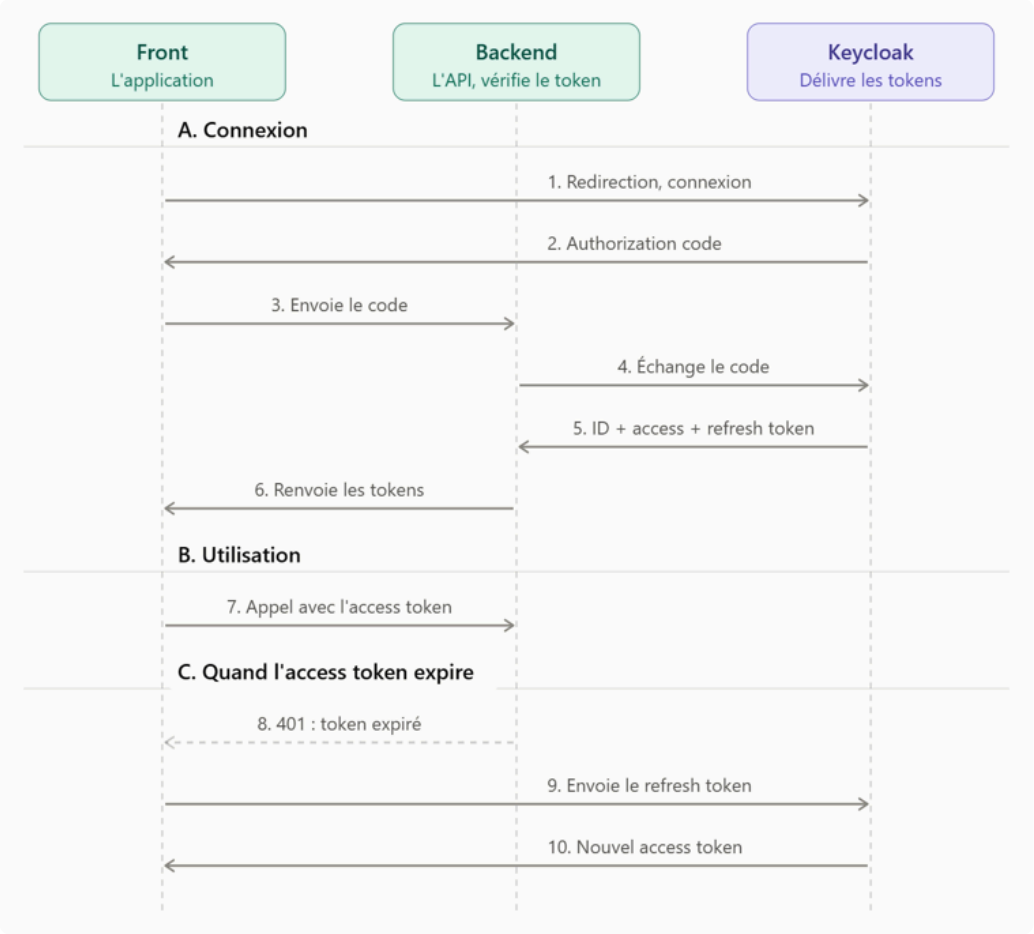
🔗 **Et ensuite l'utiliser :**

```
http.oauth2ResourceServer(oauth2 ->
    oauth2.jwt(jwt ->
jwt.jwtAuthenticationConverter(jwtAuthenticationConverter()))
);
```

Flux de OAUTH2:

Flux	Pour qui	Principe
Authorization Code	Un utilisateur qui se connecte, avec une application côté serveur	Redirection vers Keycloak, retour avec un code à usage unique, échangé contre un token
Code + PKCE	Un utilisateur, avec une application qui **ne peut pas garder de secret** (front React, mobile)	Même flux, avec une protection supplémentaire contre le vol du code
Client Credentials	**Aucun utilisateur** : un service qui parle à un autre	Le service s'authentifie avec son identifiant et son secret, et obtient un token pour lui-même
Refresh Token		not a login flow; exchanges a refresh token for a new access token when the old one expires

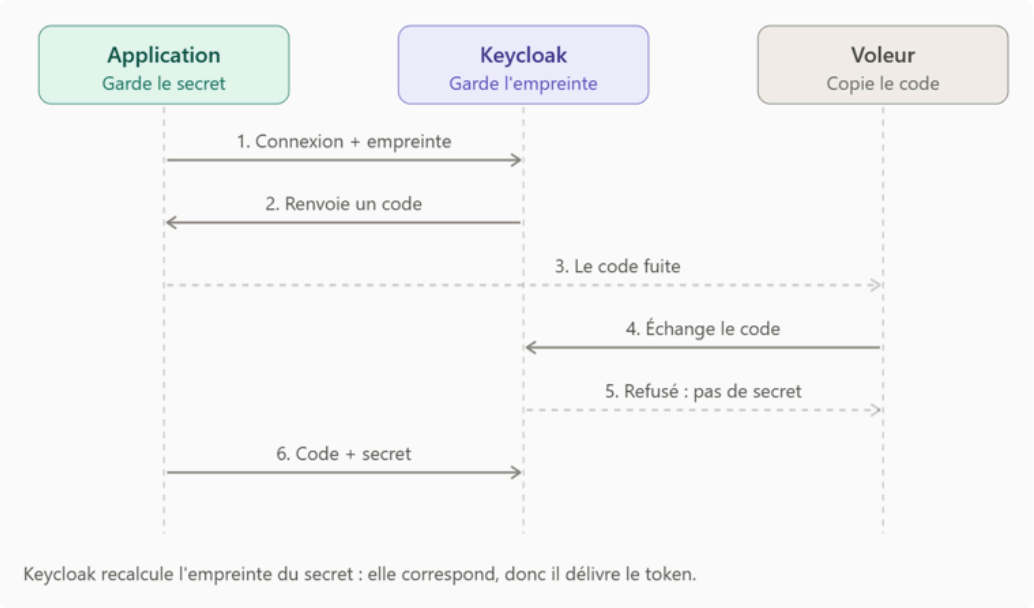
Authorisation code:



Comment lire ce diagramme

- **A. Connexion (étapes 1 à 6)** : le front est redirigé vers Keycloak, qui renvoie un **authorization code**. Le backend l'échange contre trois tokens : **ID token, access token et refresh token**.
- **B. Utilisation (étape 7)** : le front appelle l'API avec l'**access token**.
- **C. Expiration (étapes 8 à 10)** : l'API répond 401, le front envoie le **refresh token** à Keycloak et reçoit un **nouvel access token**, sans nouvelle connexion.

Authorisation Code + PKCE



1. L'application lance la connexion et envoie l'**empreinte**. Keycloak la garde.
2. Keycloak renvoie un **code**.
3. Le code fuite : le voleur le copie.
4. Le voleur essaie d'échanger le code.
5. **Refusé**, car il n'a pas le secret.
6. L'application envoie le code et le **secret** : Keycloak accepte.

Le secret et l'empreinte

Ce sont les deux mêmes données, sous deux formes :

Dans le schéma	Nom technique	Ce que c'est
Secret	`code_verifier`	Une chaîne aléatoire créée par l'application à chaque connexion
Empreinte	`code_challenge`	Le résultat d'un calcul SHA-256 sur le secret

- Au début, l'app envoie l'**empreinte hashée** (code_challenge).
- À la fin, l'app envoie le **secret en clair** (code_verifier).

Pourquoi un voleur échoue :

- Il vole le **code**, mais il n'a pas le **secret**.
- Il voit l'**empreinte** dans l'URL, mais le hash est à sens unique, donc il ne peut pas retrouver le secret.
- Sans le secret, Keycloak refuse.

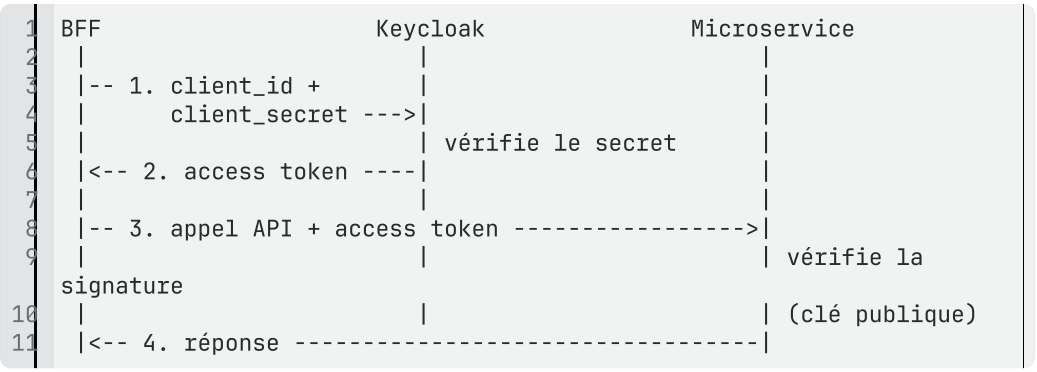
Client Credentials

Pas **d'utilisateur, une machine parle à une machine.**

Idée en une phrase : un service s'identifie lui-même auprès de Keycloak avec son `client_id` + `client_secret`, et reçoit directement un **access token**. Il n'y a ni login, ni authorization code, ni ID token, ni refresh token.

Analogie : un badge d'employé pour un robot. Le robot montre son badge (id + secret) et reçoit un laissez-passer. Personne ne se connecte.

Schéma de séquence (cas BFF → microservice) :



Règle clé : le `client_secret` reste **uniquement côté backend**. Jamais dans React.

Deux choses différentes :

	Pour quoi faire	Qui l'utilise
client_id + client_secret	**Obtenir** un token (se présenter à Keycloak)	Le BFF, quand il demande un token à Keycloak
Clé publique de Keycloak	**Vérifier** un token reçu	Le BFF et les microservices

Comment le BFF valide un JWT (les microservices font exactement pareil) :

1. **Signature** : le JWT est signé par Keycloak. On la vérifie avec la **clé publique** de Keycloak. Si le token est modifié, la signature ne correspond plus.
2. **Expiration** (`exp`) : le token n'est pas expiré.
3. **Émetteur** (`iss`) : le token vient bien de **notre** Keycloak.
4. **Destinataire** (`aud`), si configuré : le token est bien prévu pour cette API.

> **Où est la clé publique ?** Les services la récupèrent une fois sur Keycloak (endpoint **JWKS**, via `issuer-uri` dans Spring) et la gardent en cache. Ensuite il n'y a plus d'appel à Keycloak à chaque requête.
>
> **Ce que le microservice fait en plus :** après avoir validé le token, il lit les **rôles et droits** dedans et décide si l'utilisateur peut faire l'action (`@PreAuthorize` , habilitations). Le BFF valide, mais c'est le service qui protège les données.

- Le JWT a 3 parties : **header . payload . signature**.
- Si quelqu'un modifie le payload (`user` → `admin`), la signature ne correspond plus.
- Le BFF et le service le détectent avec la clé publique.

Avant : clé publique en dur dans le projet

- ❌ Si Keycloak change de clé (rotation, incident de sécurité), tous les tokens valides sont **rejetés**, et il faut modifier et redéployer chaque service.
- ❌ Avec 10 microservices, il faut mettre à jour 10 endroits.

Maintenant : récupération via l'issuer (JWKS)

- Le service lit la configuration chez Keycloak et récupère **automatiquement** les clés publiques.
- La **rotation** des clés ne casse rien.
- Rien à copier à la main.

Et OpenID Connect alors ?

- **OIDC = OAuth2 + une couche d'identité**
- Ajoute un **ID Token** pour que le client connaisse l'utilisateur connecté
- L'app veut savoir **qui est connecté** → elle lit l'**ID token**.
- L'API veut savoir **ce qu'il a le droit de faire** → elle lit l'**access token**.

« L'ID token dit qui est l'utilisateur, il est destiné à l'application. L'access token donne l'accès à l'API. »

OAuth2	OpenID Connect
Permet juste l'autorisation	Ajoute l'authentification (login)
Pas d'infos sur l'utilisateur	Contient un ID Token avec infos user
→ accès aux ressources	→ + qui est l'utilisateur ?

In Keycloak, roles are stored in `realm_access.roles` or `resource_access`. These roles need to be converted to Spring Security authorities using a `JwtAuthenticationConverter`, which maps them to `ROLE_` prefixed `GrantedAuthority` objects. This way, Spring can enforce access control using annotations like `@PreAuthorize("hasRole('ADMIN')")`

To secure a Spring Boot REST API with Keycloak, I configure Spring Security as a resource server using the `spring-boot-starter-oauth2-resource-server`

dependency and set the `issuer-uri` so Spring can validate the JWT issued by Keycloak.
I then define a `SecurityFilterChain` bean and a custom `JwtAuthenticationConverter` to extract roles from the token (`realm_access.roles`) and map them to `ROLE_XXX` so Spring Security can recognize them as authorities.
With this setup, I can use annotations like `@PreAuthorize("hasRole('ADMIN')")` to secure endpoints.

Symétrique / Asymétrique

	Symétrique (HS256)	**Asymétrique (RS256)**
Nombre de clés	1 clé secrète	2 clés : privée + publique
Pour signer	La clé secrète	La clé **privée**
Pour vérifier	La même clé secrète	La clé **publique**
Qui peut créer un token ?	Tous ceux qui ont la clé	Seulement celui qui a la clé privée
Si un service est compromis	Faux tokens possibles ⚠️	Seulement la clé publique est exposée, donc pas de faux tokens ✅
Partage de la clé	Doit rester secrète partout	La publique peut être partagée
Vitesse / complexité	Rapide, simple	Un peu plus lent et complexe
Quand l'utiliser	Un seul système signe et vérifie	Un émetteur, plusieurs vérifieurs
Analogie	Cadenas, une clé pour tous	Tampon officiel, vérifiable par tous
Dans tes projets	Lien de réinitialisation	Tokens **Keycloak** (BFF et services)